DSA4213 Assignment 1: Does Better Next-Token Prediction Give Better Representations?

Zhongze Zhao (A0357793J)

Code and `README.md`: `https://github.com/zzz84402816-lab/DSA4213-Assignment-1` (also submitted as a zip)

1 Question and setup

Comparison question. I investigate whether stronger next-token prediction leads to more useful features for topic classification. Part I compares four language-model (LM) families using perplexity (PPL). Part II keeps the LSTM architecture fixed and varies its input embeddings. Part III evaluates frozen representations on AG News and compares the downstream ranking with the language-model ranking.

Corpus and preprocessing. I use the official WikiText-2 v1 splits (`wikitext-2-v1`), containing approximately 2.08M training, 216k validation and 244k test tokens. The release is already tokenised and replaces rare words with `<unk>`. I lowercase the text for compatibility with uncased GloVe, remove empty lines and add `<eos>` after each line. All models share a 28,913-entry vocabulary constructed only from training tokens, including `<pad>`, `<unk>` and `<eos>`. Lowercasing introduces no additional validation or test OOVs. The `<unk>` token accounts for 6.2% of test tokens and is included in evaluation.

Common evaluation. All models follow the same evaluation protocol with non-overlapping 128-token windows and state resets between windows. A prediction uses only the available preceding tokens within its window. There are 244,101 test targets, with every test token after the first scored once. I right-pad the final window and exclude padding targets from the loss. I report average cross-entropy (CE) in nats per non-padding token and perplexity, calculated as $\mathrm{PPL} = \exp(\mathrm{CE})$. Checkpoints are selected using validation loss, and the selected model is evaluated once on the test split.

Models. I compare four model families. The n-gram baseline is my interpolated Kneser–Ney implementation, using an order-specific discount $D_k = n_1/(n_1 + 2n_2)$, lower-order continuation counts and a uniform base distribution. Validation PPL was 274.8, 235.8 and 229.3 for orders 2, 3 and 4, so I selected the 4-gram. The RNN and LSTM each use 300-dimensional embeddings, two 512-unit layers and a linear softmax output. The decoder-only Transformer uses pre-layer normalisation, four layers, six heads, hidden size 300, FFN size 1200 and learned positions. Its causal mask blocks future positions. A unit test for all three neural models verifies that changing tokens at or after position t does not affect earlier logits. The neural models share the same training settings: trainable 300-dimensional embeddings initialised from $\mathcal{N}(0, 1)$, dropout 0.3, context length 128 and batch size 32. Each run lasts 20 epochs, approximately 10.1k steps or 41.5M target tokens. I use AdamW with learning rate 10^{-3} and weight decay 0.01, followed by cosine decay after 300 warm-up steps, and clip gradients at 1.0. The checkpoint with the lowest validation loss is retained. The primary runs use seed 42 and PyTorch 2.11 on a 12 GB RTX 5070 Ti Laptop GPU under WSL2.

2 Part I: four language-model families

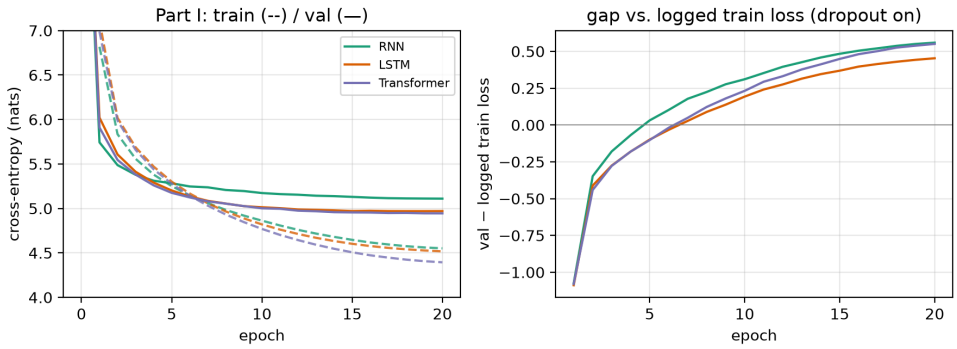


Figure 1: Part I training (dashed) and validation (solid) cross-entropy per epoch, and their difference. The logged training loss is averaged over the epoch while the weights change and with dropout on, so it over-states the true training loss: this is why the difference is *negative* early on and why it under-states the final gap (clean gaps at the selected checkpoint are in Table 1).

Model	Test CE (nats)	Test PPL seed 42	Test PPL 3 seeds	Train CE (eval mode)	Val CE	Gap val−train
KN 4-gram	5.378	216.5	(deterministic)	2.10	5.44	3.33
RNN	5.033	153.4	153.2 ± 0.2	4.13	5.11	0.98
LSTM	4.905	135.0	135.0 ± 0.7	4.26	4.97	0.70
Transformer	4.876	131.0	131.8 ± 0.6	3.84	4.94	1.11

Model	Params (M)	Non-emb (M)	Train time s (s/epoch)	Train k tok/s	Infer k tok/s	Gen ms/token recomp.	Gen ms/token increm.	Rep-3 greedy	Dist-2 top-k
KN 4-gram	5.8[†]	–	7 (CPU)	–	450 (CPU)	–	0.45 (CPU)	0.55	0.93
RNN	24.4	0.94	433 (22/ep)	105	274	2.82	0.43	0.34	0.92
LSTM	27.3	3.77	449 (22/ep)	101	273	2.98	0.49	0.43	0.92
Transformer	21.8	4.37	384 (19/ep)	126	484	1.36	1.51	0.41	0.90

Table 1: Part I. Top: prediction quality and fit. *3 seeds*: mean ± std of test PPL over seeds 42, 1, 2. *Train CE*: training split re-scored in eval mode (no dropout) with the selected weights, so that *Gap* = val CE − train CE compares like with like. Bottom: cost and generation. *Train time*: full 20-epoch budget, all three models re-trained in lockstep (alternating order per epoch) so they share the same thermal conditions. *Train/Infer k tok/s*: round-robin benchmark (5 rounds, median; batch 32×128). *Gen*: model-only greedy decoding at batch 1, 120 new tokens, median of 5 round-robin rounds; *recomp.* re-encodes the whole context each step, *increm.* carries the recurrent state or a key/value cache. *Rep-3*: share of repeated trigrams in 40-token greedy continuations; *Dist-2*: distinct bigrams in top-k samples (5 prompts ×10). [†]stored n-gram count entries, not trainable parameters.

Prediction quality. Observation. The neural models all achieved substantially lower test PPL than the KN 4-gram. Across three seeds, the order was Transformer (131.8 ± 0.6), LSTM (135.0 ± 0.7), then RNN (153.2 ± 0.2), compared with 216.5 for the 4-gram. The Transformer had a small advantage over the LSTM (Welch t-test p=0.004), while both were clearly ahead of the RNN. **Possible explanation.** The 4-gram relies on the previous three tokens and must back off when a context is unseen. Neural models can share information through embeddings and access longer contexts. The RNN's lack of gates may make long-range information harder to preserve. However, it also has fewer non-embedding parameters (0.94M versus about 3.8M and 4.4M), so this comparison does not separate architecture from capacity.

Learning behaviour. Observation. Figure 1 shows that the RNN improved quickly at first but reached a plateau sooner than the other neural models. The LSTM's validation curve became nearly flat after about 12 epochs, with its best checkpoint at epoch 19; the Transformer was still improving slightly at epoch 20. None showed an upward validation trend. Nevertheless, evaluation-mode scoring revealed substantial val−train CE gaps (Table 1): 1.11 nats for the Transformer, 0.98 for the RNN, 0.70 for the LSTM and 3.33 for the 4-gram. These gaps varied by at most 0.02 across seeds. **Possible explanation.** The models fit training-specific patterns that do not transfer fully to unseen text. Dropout and the learning-rate schedule may have helped, but their effects were not tested separately. The Transformer combined the lowest validation loss with a larger gap than the LSTM, showing that gap size alone does not determine test quality. Its unfinished learning curve suggests that more training could help; the RNN's larger gap may reflect overfitting to local patterns in the training data or hyperparameter settings that are less suitable for this architecture, although neither explanation has been verified.

Cost. Observation. Sequential training runs gave misleading wall-clock times on the laptop GPU (epoch times of the *same* model drifted from 21 to 33 s as it heated up), so I measured cost under shared conditions. Re-training the three models in lockstep for the full budget, the Transformer is the fastest to train (384 s vs. 433 s RNN and 449 s LSTM; it reaches the same validation loss as before), and the round-robin benchmark agrees (training 126k vs. 101–105k tok/s; scoring 484k vs. 273–274k tok/s). The KN model trains in 7 s on one CPU thread. *Generation* reverses the ranking. Re-encoding the context at every step, the RNN/LSTM cost grows with length (1.60 → 2.82 ms/token for the RNN from 40 to 120 tokens) while the Transformer stays at 1.28–1.36 ms. Carrying the recurrent state makes the RNN/LSTM constant at ≈0.43–0.49 ms/token, 2.5–3× faster than the Transformer, whose key/value cache (1.51 ms) brings no gain at this scale. **Possible explanation.** Training and scoring see the whole window at once, which the Transformer parallelises across positions while recurrent models must step through it; generation produces one token at a time, where an $O(1)$ recurrent update is cheap. At batch 1 and ≤128 tokens the GPU is mostly idle and latency is dominated by per-operation launch overhead, so my unfused cache cannot beat PyTorch's fused full-context kernel; a fused cache or longer contexts could change the Transformer's numbers, which I did not test.

Generated text. Observation. With matched prompts and 40-token continuations (Table 2; `<unk>` banned), greedy decoding caused repetition in every model. The repeated-trigram rate was highest for KN (55%) and lowest for the RNN (34%). Top-k sampling greatly reduced this behaviour. Some neural samples contain recognisable clauses and military-related words, whereas the 4-gram often connects short, locally plausible

fragments. However, grammatical wording does not guarantee sensible content: *"the royal fleet was an apprentice officer"* is semantically inconsistent. Topic continuity is also unreliable beyond the opening sentence. The neural models reproduce WikiText formatting, including headings and `<eos>`. Overall, these samples suggest better local structure than sustained, meaningful discourse. *Blind rating.* I also scored 10 randomly selected top-k samples from each model after shuffling them and hiding model identities (Table 3). Fluency and consistency used the 1–3 rubric in `README.md`. No sample received the highest fluency score. Among Part I models, the Transformer had the highest mean consistency (1.70), while every 4-gram sample scored 1. Their difference was significant (Mann–Whitney p=0.005), but Transformer versus LSTM was not (p=0.07). Mean fluency scores of 1.2–1.6 did not establish a clear ordering. **Possible explanation.** Better next-token prediction may improve short-range continuity without ensuring discourse coherence. Limited data or capacity may contribute, but this experiment does not identify the cause. The reduction in repetition under top-k sampling indicates that decoding choice also matters. These conclusions remain tentative because there was one rater and only 10 samples per model.

Prompt: *during the second world war ,* (top-k=40, T=0.8 unless noted; first sample, first 24 tokens)	
KN 4-gram	and , unlike the most controversial of all the millennium stadium , but i just want to be with them , and the first . . .
RNN	and did not have two different types of the solar family . ⟨eos⟩in the late 20th century , many archaeologists were not only . . .
LSTM	the royal fleet was an apprentice officer on the main naval brigade . he was appointed adjutant 's flagship by the british navy , . . .
Transformer	the company took place off their first position in australia , where they suffered a new zealand brigade , they had been unable to . . .
LSTM+Skip-gram	the storm quickly weakened northeast across the united states over the west coast , and the storm eventually moved across the atlantic ocean . . . .
LSTM+GloVe	the largest had been estimated and was considered a high @-@ time @-@ known @-@ scale @-@ time @-@ long area . the first . . .
KN 4-gram (greedy)	and the first to be released in the united states , and the first to be released in the united states , and the . . .
Transformer (greedy)	the first world war ii was the first world war ii . the war was the first world war ii in the war , . . .

Table 2: Matched continuations (more in `results/generations.md`).

Model	Test PPL	Fluency	Consistency
KN 4-gram	216.5	1.20 [1.0, 1.5]	1.00 [1.0, 1.0]
RNN	153.4	1.30 [1.0, 1.6]	1.30 [1.1, 1.6]
LSTM	135.0	1.30 [1.0, 1.6]	1.20 [1.0, 1.5]
Transformer	131.0	1.50 [1.2, 1.8]	1.70 [1.3, 2.1]
LSTM + Skip-gram	130.4	1.60 [1.3, 1.9]	1.70 [1.3, 2.1]
LSTM + GloVe	133.9	1.30 [1.0, 1.6]	1.20 [1.0, 1.5]

Table 3: Blind human rating of top-k samples (one rater, 10 samples per model, 1–3 scale): mean and bootstrap 95% CI.

3 Part II: input-embedding ablation (LSTM)

Design. I vary the input embeddings of the LSTM, the stronger recurrent model in Part I. Setting 1 reuses the Part I configuration with trainable random embeddings. Setting 2 uses frozen 300-dimensional Skip-gram vectors trained only on WikiText-2's training split in gensim, with window size 5, five negative samples and 10 epochs. They cover the full vocabulary and require 82 s on one CPU thread, excluded from LM training time. Setting 3 uses frozen GloVe 6B 300d vectors from Wikipedia 2014 and Gigaword 5. After the mappings `@-@`→`-`, `@,@`→`,` and `'t`→`n't`, coverage is 96.2% of vocabulary types and 95.9% of training tokens. The 1,098 uncovered types receive random $\mathcal{N}(0, \sigma^2)$ vectors, where σ is GloVe's standard deviation; `<pad>` is zero. These rows are frozen too. All remaining parameters are trained from scratch using the Part I procedure. In addition to seed 42, I repeat the runs with seeds 1 and 2.

Input embedding	Cover.	Train. (M)	Test PPL seed 42	Test PPL 3 seeds	Val PPL ep 1	Val PPL ep 3	Ep. to best+.05	Gap val−train	Step-loss std
Random $\mathcal{N}(0,1)$, trainable	–	27.3	135.0	135.0 ± 0.7	412	224	10	0.70	0.127
Own Skip-gram, frozen	100.0%	18.6	130.4	130.2 ± 1.9	559	239	11	0.64	0.123
GloVe 6B, frozen	96.2%	18.6	133.9	132.7 ± 1.1	564	237	11	0.57	0.122
Suppl.: random $\mathcal{N}(0, 0.1^2)$, train.	–	27.3	140.3	–	706	274	11	0.71	0.123

Table 4: Part II. *Cover.*: vocabulary types with a pretrained vector. *3 seeds*: mean ± std over seeds 42, 1, 2. *Ep. to best+.05*: first epoch whose val loss is within 0.05 nats of the run's best. *Gap*: val − train CE at the selected checkpoint, both scored in eval mode (as in Table 1). *Step-loss std*: mean within-epoch std of mini-batch loss.

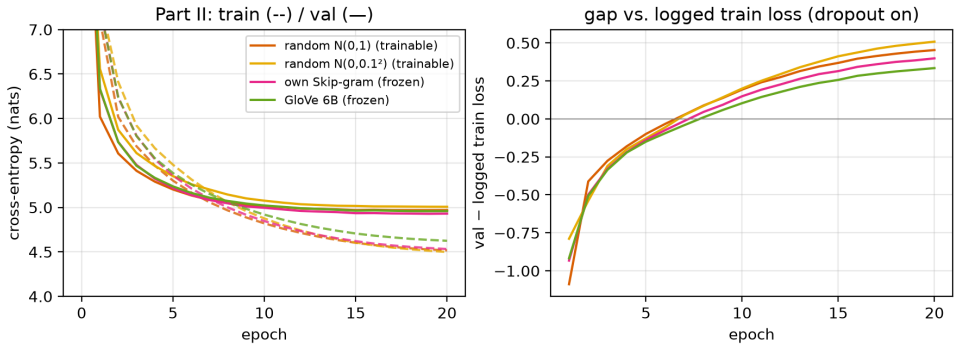


Figure 2: Part II learning curves (all LSTM, identical except the input-embedding table). As in Fig. 1, the right panel uses the logged (dropout-on) training loss; clean gaps are in Table 4.

Results. Observation. (Table 4, Fig. 2) Across three seeds, frozen Skip-gram achieved the best mean test PPL (130.2 ± 1.9), followed by frozen GloVe (132.7 ± 1.1) and trainable random embeddings (135.0 ± 0.7). The mean improvements over the random baseline were 4.8 and 2.3 PPL, respectively; the evidence is limited to three runs per setting. The random baseline improved faster initially (epoch-1 validation PPL 412 versus about 560), but the frozen settings caught up around epoch 6. All reached within 0.05 nats of their best validation loss by epoch 10–11. Step-loss variability (0.122–0.127) and maximum gradient norms (about 1.8) gave no clear stability advantage. GloVe had the smallest evaluation-mode val−train gap (0.57 versus 0.64 and 0.70 nats), together with the highest training loss. This is consistent with constrained fitting, but a smaller gap alone does not imply better prediction: Skip-gram still had the lowest test PPL. The generation metrics were similar across embedding settings: distinct-2 was 0.92, 0.91 and 0.90, and greedy repeated-trigram rates were 0.43, 0.49 and 0.47 for random, Skip-gram and GloVe. None reliably maintained coherent content. The `@-@` chain in the displayed GloVe sample (Table 2) was not unique to GloVe: among the 50 samples per model, such chains occurred 4 times for GloVe, 4 for Skip-gram, 0 for the random-embedding LSTM and 3 for the RNN. These counts do not establish whether embedding choice affects the behaviour. In the blind ratings (Table 3), Skip-gram had higher mean consistency (1.70) than GloVe or the random baseline (both 1.20), but the Skip-gram–GloVe comparison was not significant (p=0.07). With 10 rated samples per model, I treat this as a tentative pattern rather than a reliable ranking.

Why does "trainable" not win? Observation. The default trainable embeddings changed little in direction: the mean initial-to-final cosine was 0.98 for all three neural models, and 0.94 for the LSTM's 1,000 most frequent words. Their nearest neighbours remained largely unrelated in meaning, unlike those from Skip-gram or GloVe (Appendix Table 6). **Possible explanation.** The initial vectors had large norms, about 17, which may have made the learned changes small relative to the starting vectors. However, the learning rate alone does not determine the accumulated AdamW update, and cosine similarity measures direction rather than total change. The evidence therefore supports limited semantic reorganisation, not the claim that the embeddings were effectively unchanged. Adaptation may also occur in later layers. Part III further shows that even these small directional changes can improve topic features. *Supplementary test:* with a smaller initialisation, $\mathcal{N}(0, 0.1^2)$, the embeddings changed more in direction (mean cosine 0.67; 0.56 for frequent words) and developed some meaningful neighbours, such as *king–queen* and *football–rugby*. This did not improve language modelling within 20 epochs: test PPL increased to 140.3. Training CE was also higher (4.30 versus 4.26), while the val−train gap was almost unchanged (0.71 versus 0.70). Together with slower early learning (epoch-1 validation PPL 706 versus 412) and continued improvement at epoch 20, this pattern is more consistent with slower optimisation than increased overfitting. Smaller inputs may contribute, but that mechanism was not directly tested. A longer run might help, although this remains unverified. Despite its worse PPL, this model produced better downstream features in Part III.

Interpretation. The random and pretrained settings differ in both embedding training history and whether the table is updated during LM training. Comparing frozen Skip-gram with frozen GloVe holds trainability fixed, but it does not isolate a single cause: the tables also differ in learning method, corpus size, domain and coverage. In these runs, in-domain Skip-gram gave lower WikiText-2 PPL than GloVe despite using far less text. Thus, more pretraining data did not guarantee better performance on this LM objective. GloVe performed better on AG News in Part III, but that result cannot be attributed to data quantity alone. *Leakage check:* because WikiText-2 and part of GloVe's pretraining corpus both come from Wikipedia, overlap with the WikiText-2 test articles is possible. The experiment does not verify the exact overlap. Frozen word vectors encode corpus statistics rather than explicit test sequences, but this does not rule out information from overlapping data. GloVe's failure to outperform train-only Skip-gram also does not prove that overlap had no effect. I therefore treat the GloVe setting as potentially affected by pretraining overlap. The Skip-gram setting provides the cleaner comparison because its vectors were trained only on the designated WikiText-2 training split.

4 Part III: frozen LM representations for AG News

Task and data. The downstream task is four-class AG News topic classification: World, Sports, Business and Sci-Tech. I sample 20,000 training articles with equal class counts and evaluate on all 7,600 official test articles. A rule-based tokenizer follows WikiText conventions, including forms such as `didn 't` and `3 @.@ 5`. Using the WikiText-2 vocabulary, the AG News test-token OOV rate is 7.1%, close to SST-2's 7.0%, so this coverage measure did not determine the choice. I chose AG News because its register is reasonably close to Wikipedia and it provides a large labelled test set; SST-2's test labels are not public.

Representations. For each frozen LM, I truncate articles to 128 tokens and right-pad shorter inputs. I extract three representations: *Last* uses the top-layer state at the final non-padding token; *Mean* averages top-layer states over non-padding positions; *Emb* averages the input embeddings without contextual processing. Emb serves as a reference for assessing what the contextual layers contribute. Each representation uses the same standardisation and multinomial logistic-regression pipeline. I select C from $\{10^{-3}, \ldots, 1\}$ using macro-F1 on a 10% validation portion of the training subset, then refit on all 20,000 training examples. The language models remain frozen throughout.

Frozen LM	Seeds	LM test PPL	Last state test accuracy (mean ± std over LM seeds)	Mean of states test accuracy (mean ± std over LM seeds)	Mean of input emb. test accuracy (mean ± std over LM seeds)
RNN	3	153.2	0.616 ±0.001	0.713 ±0.006	0.759 ±0.004
LSTM	3	135.0	0.650 ±0.010	0.710 ±0.008	0.740 ±0.006
Transformer	3	131.8	0.730 ±0.010	0.772 ±0.007	0.772 ±0.004
LSTM + Skip-gram (frozen)	3	130.2	0.767 ±0.009	0.802 ±0.003	0.824‡
LSTM + GloVe (frozen)	3	132.7	0.794 ±0.004	0.844 ±0.002	0.878‡
Suppl.: LSTM, $\mathcal{N}(0, 0.1^2)$ init	1	140.3	0.644	0.731	0.765
Control: untrained LSTM	–	28,975	0.475	0.718	0.683
Control: untrained Transformer	–	32,833	0.496	0.650	0.683
Reference: TF-IDF bag of words, no LM	–	–		0.885	

Table 5: Part III test accuracy on 7,600 AG News test articles (macro-F1 is within 0.002 of accuracy in every cell). Trained LMs: mean ± std over the three LM seeds used in Parts I–II. ‡frozen pretrained table, hence identical for all seeds. *Controls*: the seed-42 *initial* weights of the LSTM/Transformer, never trained (both share the same initial embedding table). With n=7,600 the standard error of an accuracy near 0.8 is ≈0.005; exact McNemar tests (seed-42 models) are in `results/downstream/error_stats.json`.

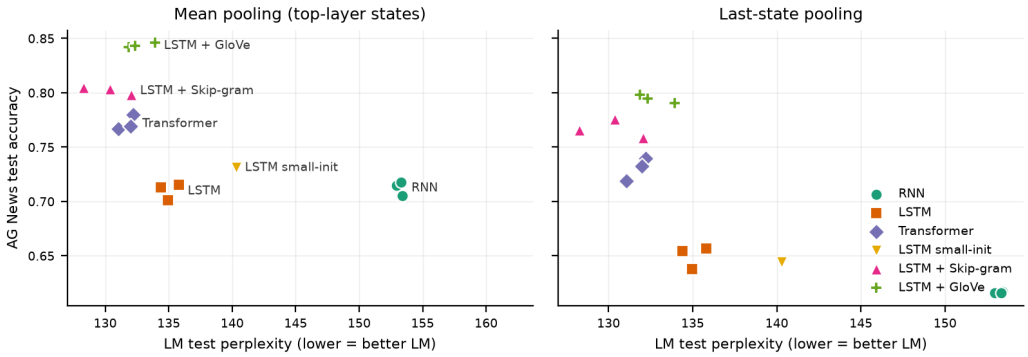


Figure 3: LM test perplexity vs. downstream accuracy, one point per trained checkpoint (3 seeds per family).

Which representation works better? Observation. (Table 5, Fig. 3) Mean pooling outperformed the last state for every trained LM; for the LSTM, accuracy was 0.710 versus 0.650 (seed-42 McNemar $p < 10^{-28}$). Among the Part I models, the Transformer had the strongest mean-pooled features (0.772 versus 0.710 for the LSTM; seed-42 $p < 10^{-37}$). The RNN and LSTM had similar accuracy despite an 18-point PPL difference. Frozen pretrained embeddings improved the LSTM results to 0.802 with Skip-gram and 0.844 with GloVe. The single small-init run also improved on the default LSTM (0.731 versus 0.710 ± 0.008), despite worse PPL. However, averaging input embeddings matched or exceeded averaging contextual states in every trained model; GloVe reached 0.878 versus 0.844. TF-IDF was numerically highest at 0.885. For this topic task, these results suggest that lexical information is particularly useful and that contextual processing does not automatically improve a pooled representation.

Does LM training help the features at all? Observation. Compared with the untrained controls (test PPL 28,975 / 32,833), training improved the last-state features substantially: accuracy rose from 0.475 to 0.650 for the LSTM and from 0.496 to 0.730 for the Transformer. The Transformer's mean-pooled accuracy rose from

0.650 to 0.772. Input-embedding averages also improved, from 0.683 to 0.740–0.772, so the small directional drift in Part II was still useful for topic classification. In contrast, the untrained LSTM's mean pooling reached 0.718, close to the trained result of 0.710 ± 0.008. **Possible explanation.** Averaging random LSTM states may preserve enough lexical information for a linear classifier to recognise topics. Next-token training need not improve that particular summary. This is a possible mechanism, not evidence that random LSTM states are literally a linear projection of word counts. The untrained Transformer mean was weaker than its embedding average (0.650 versus 0.683), suggesting less accessible lexical information after random contextual processing. Pretrained vectors and GloVe's news-domain exposure offer another possible source of useful topic information.

Does PPL rank agree with downstream rank? Observation. Across 16 trained checkpoints, lower PPL was associated with higher accuracy: Spearman $\rho = -0.66$ for mean pooling (p=0.006) and $\rho = -0.76$ for the last state (p=7×10^{-4}). The six seed-42 models alone gave weaker statistical evidence ($\rho = -0.7$, p=0.16). Within Part I, the association was clearer for the last state ($\rho = -0.82$, p=0.007) than for mean pooling ($\rho = -0.57$, p=0.11). Nevertheless, PPL did not consistently rank individual settings. RNN and LSTM had similar mean-pooled accuracy despite an 18-point PPL gap; small initialisation improved accuracy while worsening PPL; and GloVe improved accuracy by about four percentage points over Skip-gram despite PPL being 2.5 higher. These counterexamples show that PPL is not a sufficient selection criterion for this downstream task. Because the checkpoints include repeated seeds within a few model settings, the correlation should be read as a pattern in this experiment, not a universal relationship.

Error analysis. Observation. In the seed-42 results, Business and Sci/Tech were the main source of confusion. They accounted for 515 of the GloVe-LSTM's 1,170 errors. Headlines about technology companies' finances, such as Dell leaving a PC market or IBM reporting revenue growth, plausibly fit both topics. All five main LM systems misclassified the same 532 articles, including some with debatable labels. Cases corrected by GloVe often contained news-related proper nouns such as *Isinbayeva*, *DaimlerChrysler*, *Linux* and *Dalai Lama*. These examples suggest a possible vocabulary advantage, but do not establish which words caused the prediction. GloVe-LSTM accuracy was 0.82 in the shortest-length quartile and 0.85–0.86 in the others. Accuracy did not decrease with a higher `<unk>` share (Appendix Table 7). Remaining words may provide enough topic information, although differences between the article groups could also explain this pattern. Business had the lowest recall for every model.

5 Connecting the parts, limitations, and lessons

Answer to the question. Better next-token prediction was associated with better representations overall, but it was not a reliable rule for choosing features. Across 16 trained checkpoints, mean-pooled accuracy correlated negatively with PPL ($\rho = -0.66$), and the Transformer led the random-embedding models on both measures. However, RNN versus LSTM, small versus default initialisation, and GloVe versus Skip-gram all showed that a PPL improvement need not translate into an accuracy improvement. The strong embedding-average and TF-IDF results suggest that lexical information is especially useful for AG News. LM training improved several representations, but added little to the LSTM's mean-pooled accuracy in the tested setup. GloVe's downstream advantage may reflect pretraining data and domain match, which this experiment cannot separate. The practical conclusion is to evaluate features on the downstream task directly. In the controlled neural-model benchmark, the Transformer also had the highest training and scoring throughput, although the recurrent models generated faster one token at a time.

Limitations. The findings are limited to one roughly 2M-token LM corpus and a 20-epoch budget. The Transformer's continuing improvement suggests that more training could change the results, but does not establish how the ranking would change. All Part I neural models used $\mathcal{N}(0, 1)$ embeddings and showed little directional drift; using the same initialisation does not guarantee an equal effect on different architectures. The main neural settings used three seeds, while the small-init experiment and untrained controls used one. Laptop thermal conditions also limit timing precision, despite the lockstep and round-robin protocols. Downstream evaluation used frozen features and a linear classifier on AG News, so other tasks or fine-tuning may give different rankings. Human evaluation involved one rater and 10 samples per model, limiting the strength of small differences. Finally, GloVe differs in data quantity, domain and training method. Its news corpus overlaps AG News's period and domain, and its Wikipedia corpus may overlap WikiText-2; exact article overlap was not checked.

Lessons. This experiment taught me to compare computational cost under matched measurement conditions and to calculate the generalisation gap using training and validation losses scored in the same mode. Re-scoring the training set changed the interpretation of the small-init run. I also learned to inspect how trainable embeddings change, rather than treating the trainable label as evidence of useful learning. Most importantly, representation quality needs its own downstream evaluation: a lower language-model PPL is informative, but is not enough to select the best features.

Appendix

Word	LSTM, $\mathcal{N}(0,1)$ init	LSTM, $\mathcal{N}(0,0.1^2)$ init	Skip-gram (own)	GloVe 6B
king	posse, designer, bishop, schoolmaster	queen, bishop, governor, emperor	veera, æthelred, leinster, ballala	queen, prince, monarch, kingdom
war	mart, [non-Latin] kalachuris, 1850	fastra, ahk, years, year	outbreak, civil, liberties, airbase	wars, conflict, battle, civil
football	ace, universal, joker, jallon	rugby, premier, hockey, basketball	athletic, f.c., premier, varsity	soccer, basketball, league, baseball
france	outreach, blazers, chivo, cups	parishes, supporter, galleys, steffy	italy, germany, sweden, russia	french, paris, belgium, spain
album	crush, roxas, predict, carey	episode, series, game, band	slayer, albums, yankovic, furtado	albums, song, ep, band
good	eaten, 1972, radial, outstandingly	my, everyday, homosexual, personal	sublime, shocking, exciting, naive	better, really, always, you

Table 6: Cosine nearest neighbours in four 300-d input-embedding tables (LSTM columns: tables after LM training with default and small initialisation).

`<unk>` share	0	(0,5%]	(5,10%]	>10%
Articles	838	2122	2686	1954
LSTM (mean)	0.660	0.713	0.696	0.714
Transformer (mean)	0.741	0.774	0.759	0.779
LSTM + GloVe (frozen) (mean)	0.828	0.850	0.846	0.850

Table 7: AG News test accuracy by the share of `<unk>` tokens in an article (mean pooling).

Reproduction. `scripts/run_all.sh` regenerates every number, table and figure; all tables in this report are produced by `src/latex_tables.py` from `results/`.

References, sources and tool use

Tools and sources. WikiText-2 (Merity et al., 2017), GloVe (Pennington et al., 2014), AG News (Zhang et al., 2015), SST-2 (Socher et al., 2013) via HuggingFace `datasets` / `gensim`; PyTorch, gensim, scikit-learn. Kneser–Ney follows Chen & Goodman (1999). Code structure adapted from the DSA4213 course notebooks.
AI assistance: I used Claude Code (Anthropic) as a coding assistant for code scaffolding, debugging, running experiments and drafting parts of this report, and ChatGPT (OpenAI, astra6) to help rewrite and edit the analysis text in my own words; the free-text note column of the rating sheet was filled by an AI tool from my scores and is not used in the analysis (all ratings are my own). All numbers are produced by the submitted code; I checked them against the result files and the interpretation reflects my own understanding.

References. Chen & Goodman (1999), An empirical study of smoothing techniques for language modeling, *CSL*. Merity et al. (2017), Pointer sentinel mixture models, *ICLR*. Pennington et al. (2014), GloVe, *EMNLP*. Socher et al. (2013), Recursive deep models for semantic compositionality, *EMNLP*. Zhang et al. (2015), Character-level convolutional networks for text classification, *NeurIPS*.